Build Task — Project 2: Tool-Using Agent — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python).

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

Basic Version

One agent, made only of pieces the exercises above already practiced:

- the loop, `find_tool()` / `run_tool()`, the logger, `AgentResult` and `MaxIterationsExceeded` — the Intermediate exercise, Approach 2;
- the "answers directly" check — the Edge cases exercise;
- the step limit and its log — the Failure exercise;
- Doc06's `add` and `get_weather`, copied unchanged — their Pydantic `args_schema` is the argument check.

Copy these in unchanged first: `tools.py`, `http_client.py`, `exceptions.py`, `logging_setup.py` (Doc06's Build Task).

Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

The one new piece is a tool that **fails once, then works**, so the recovery Test Case is repeatable:

- `docs_search.py`: a `FlakyToolState` class with a `call_count`, one shared `search_state = FlakyToolState()`, and `@tool search_docs(query)` that raises `ConnectionError` when `call_count == 1`.
- A test sets `search_state.call_count = 0` before the run, so the first search really fails.

`MaxIterationsExceeded` moves into `exceptions.py`, next to Doc02's named errors. `test_agent.py` has one function per Test Cases row, in the plain `case` print style of Doc04–06 — or as pytest tests, each with a fresh fail-once tool from the Intermediate exercise's factory. For your own argument checks, reuse its `run_tool_checked()` idea: an `ARG_MODELS` dict and `model_dump()`.

Difference between Basic and Intermediate: Basic lists where each piece was practiced. Intermediate designs the one new piece — the fail-once tool — and where each file's code goes.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
exceptions.py: Doc06's file + class MaxIterationsExceeded
docs_search.py: FlakyToolState, search_state, @tool search_docs
agent.py: AGENT_TOOLS = [add, get_weather, search_docs]
          AgentResult, find_tool, run_tool, run_agent(task, max_iterations)
main.py: input a task, run it, print the answer and every step
test_agent.py: the 4 Test Cases
```

Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

```
# docs_search.py — the new piece
class FlakyToolState:
    def __init__(self) -> None:
        self.call_count = 0


search_state = FlakyToolState()


@tool
def search_docs(query: str) -> str:
    """Search the company's policy documents (refunds, shipping)."""
    search_state.call_count = search_state.call_count + 1
    if search_state.call_count == 1:
        raise ConnectionError("search index temporarily unavailable")
    ...
```

`agent.py` is the Intermediate exercise's Approach 2 with `AGENT_TOOLS` instead of `ALL_TOOLS`. Finish the files, then compare against the Solution.

Difference between Basic and Intermediate: Basic is the plan. Intermediate is the new tool, almost complete.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Copied in unchanged: `tools.py`, `http_client.py`, `logging_setup.py` (Doc06's Build Task), and `exceptions.py` with one class added. Run everything from inside `practice/build_task/`.

Basic Version

Approach 1 — the loop, with Doc06's tools only

Story — Basic: the Intermediate exercise's `run_agent()` already is an agent. Run this one from inside `practice/`, next to `react_loop_practice.py`. Using it as-is, with Doc06's tools, gives answers and a step log. **If not:** there'd be nothing to test yet.

```
# main.py
from react_loop_practice import run_agent

task = input("What do you need? ")
result = run_agent(task, 5)
print("Answer:", result.final_answer)
for step in result.steps:
    print(step)
```

This answers tasks, but no tool ever fails on purpose — the "recovers from a failure" Test Case can't be checked — and a run that hits the limit crashes `main.py` with a traceback.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — a fail-once tool, a named error in `exceptions.py`, and the 4 Test Cases

Story — `exceptions.py`: Doc06's file, unchanged, plus `MaxIterationsExceeded` — the same named-error pattern as `PermanentHTTPError`, carrying the partial step log. **If not:** a run that hit the limit would look the same as a crash.

```
# exceptions.py — the class added at the end of Doc06's file
class MaxIterationsExceeded(Exception):
    """Raised when the agent loop hits its step limit, with the log."""

    def __init__(self, message: str, steps: list[dict]) -> None:
        # why: the partial step log travels with the error, so the caller
        # can see exactly what happened before it gave up
        self.steps = steps
        super().__init__(message)
```

Story — `docs_search.py`: a tool that fails on its first call, on purpose, controlled by a counter a test can reset. **If not:** tool failures would only happen by bad luck, and recovery could never be tested on demand.

```
# docs_search.py
from langchain_core.tools import tool

# why: a small, known "document store" — a test can check the agent
# really used it instead of guessing
COMPANY_DOCS = {
    "refund": "Refunds are accepted within 30 days, with the receipt.",
    "shipping": "Standard shipping takes 3-5 working days.",
}


class FlakyToolState:
    def __init__(self) -> None:
        self.call_count = 0


# how: one shared counter; a test sets it back to 0 before each run
search_state = FlakyToolState()
```

```
@tool
def search_docs(query: str) -> str:
    """Search the company's policy documents (refunds, shipping)."""
    search_state.call_count = search_state.call_count + 1
    if search_state.call_count == 1:
        # why: fails on the first call on purpose, so a run proves the
        # agent recovers instead of crashing or making up an answer
        raise ConnectionError("search index temporarily unavailable")
    lowered = query.lower()
    for topic in COMPANY_DOCS:
        if topic in lowered:
            return COMPANY_DOCS[topic]
    return "No matching document found."
```

Story — `agent.py`: the Intermediate exercise's loop (Approach 2), now with `AGENT_TOOLS` — two Doc06 tools and the fail-once search — and the same optional `tools` list, so a test can pass its own. **If not:** Project 2 would be the first place these pieces ever ran together.

```
# agent.py
from dataclasses import dataclass, field

from langchain_core.tools import BaseTool
from langchain_openai import ChatOpenAI

from docs_search import search_docs
from exceptions import MaxIterationsExceeded
from logging_setup import get_logger
from tools import add, get_weather

# why: Doc06's tools, reused unchanged, plus the one that fails once
AGENT_TOOLS = [add, get_weather, search_docs]


@dataclass
class AgentResult:
    final_answer: str
    # why: a list default needs field(default_factory=list), not `= []`
    steps: list[dict] = field(default_factory=list)


def find_tool(tools: list[BaseTool], name: str) -> BaseTool | None:
    # how: Doc06's Build Task function, copied unchanged
    for candidate in tools:
        if candidate.name == name:
            return candidate
```

```
    return None


def run_tool(
    call: dict, tools: list[BaseTool]
) -> tuple[str | None, str | None]:
    # how: returns (result, None) on success, (None, error) on failure
    tool = find_tool(tools, call["name"])
    if tool is None:
        return None, f"unknown tool: {call['name']}"
    try:
        # how: .invoke() checks the arguments with the tool's Pydantic
        # args_schema first (Doc06) — bad arguments never reach the tool
        return str(tool.invoke(call["args"])), None
    except Exception as e:
        # why: any tool failure becomes an Observation, never a crash
        return None, str(e)


def run_agent(
    task: str, max_iterations: int, tools: list[BaseTool] | None = None
) -> AgentResult:
    # why: None means "use AGENT_TOOLS"; a test can pass its own tools
    if tools is None:
        tools = AGENT_TOOLS
    logger = get_logger(__name__)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools(tools)
    messages = [{"role": "user", "content": task}]
    steps: list[dict] = []

    for step in range(max_iterations):
        response = model_with_tools.invoke(messages)
        messages.append(response)
        if not response.tool_calls:
            return AgentResult(final_answer=response.content, steps=steps)

        # why: run EVERY call the model asked for — each needs an answer
        for call in response.tool_calls:
            result, error = run_tool(call, tools)
            if error is None:
                observation = result
                logger.info(f"step {step}: {call['name']} -> {result}")
            else:
                observation = f"Error: {error}"
                # how: a warning, not an error — the loop keeps going
                logger.warning(f"step {step}: {call['name']} failed: {error}")
```

```
            messages.append({
                "role": "tool",
                "content": observation,
                "tool_call_id": call["id"],
            })
            steps.append({
                "step": step,
                "tool": call["name"],
                "arguments": call["args"],
                "result": result,
                "error": error,
            })

    message = f"No answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)
```

Story — `main.py`: reads a task, runs it, and prints the answer and every step — or, when the limit is hit, the partial log. **If not:** a stopped run would end in a traceback instead of an explanation.

```
# main.py
from agent import run_agent
from exceptions import MaxIterationsExceeded

MAX_ITERATIONS = 5


def main() -> None:
    task = input("What do you need? ")
    try:
        result = run_agent(task, MAX_ITERATIONS)
    except MaxIterationsExceeded as e:
        # why: even a failed run shows its full step log
        print(f"Gave up: {e}")
        for step in e.steps:
            print(step)
        return

    print(f"Answer: {result.final_answer}")
    for step in result.steps:
        print(step)


if __name__ == "__main__":
    main()
```

Story — `test_agent.py`: the 4 Test Cases, one function each, in the plain `case` style of Doc04–06. **If not:** "it recovers" and "it stops" would be things you checked once by hand.

```
# test_agent.py
# Runs the 4 Test Cases from the README. Run it from inside this folder:
# python test_agent.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from agent import run_agent
from docs_search import search_state
from exceptions import MaxIterationsExceeded


def test_clear_task_uses_the_tool() -> None:
    result = run_agent("What's 5 + 7?", 5)
    tools_used = []
    for step in result.steps:
        tools_used.append(step["tool"])
    print(f"case 1 (clear task)   -> tools: {tools_used}")
    print(f"                         answer: {result.final_answer}")


def test_no_tool_needed() -> None:
    result = run_agent("What's the capital of France?", 5)
    print(f"case 2 (no tool)      -> {len(result.steps)} tool calls")


def test_recovers_from_a_failure() -> None:
    # how: reset the shared counter, so the first search really fails
    search_state.call_count = 0
    result = run_agent("What is the refund policy?", 5)
    failed = 0
    succeeded = 0
    for step in result.steps:
        if step["error"] is None:
            succeeded = succeeded + 1
        else:
            failed = failed + 1
    print(f"case 3 (recovery)     -> failed={failed}, succeeded={succeeded}")
    print(f"                         answer: {result.final_answer}")


def test_step_limit() -> None:
    # how: a two-step task with room for only one step
    try:
        run_agent("Add 1 + 2, then add 3 to that.", 1)
        print("case 4 (step limit)   -> no error (wrong!)")
    except MaxIterationsExceeded as e:
```

```
        print(f"case 4 (step limit)   -> raised, {len(e.steps)} step logged")


if __name__ == "__main__":
    test_clear_task_uses_the_tool()
    test_no_tool_needed()
    test_recovers_from_a_failure()
    test_step_limit()
```

Expected output (answers vary; one `WARNING` log line appears for the failed search):

```
case 1 (clear task)   -> tools: ['add']
                         answer: 5 + 7 is 12.
case 2 (no tool)      -> 0 tool calls
case 3 (recovery)     -> failed=1, succeeded=1
                         answer: Refunds are accepted within 30 days.
case 4 (step limit)   -> raised, 1 step logged
```

Difference from Basic: a tool that fails once on command, the named error in `exceptions.py`, a `main.py` that prints the partial log instead of crashing, and all 4 Test Cases checked.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Approach 2 — pytest tests, each with a fresh fail-once tool

Story — pytest + a factory: Approach 1's shared `search_state` must be reset by hand before a test. The Intermediate exercise's factory (Approach 3) builds a new tool with its own counter for each test, and the Edge cases exercise's pytest style (Approach 2) turns each Test Case into a pass/fail `assert`. **If not:** one test's failed call could leave the counter at 1 and make the next test pass for the wrong reason.

`docs_search.py` — add the factory below `search_docs`:

```
# docs_search.py — add below search_docs
def make_flaky_search_docs(state: FlakyToolState):
    # how: the Intermediate exercise's factory (Approach 3) — every tool it
    # builds has its own counter, so tests never share one
    @tool
    def search_docs(query: str) -> str:
        """Search the company's policy documents (refunds, shipping)."""
        state.call_count = state.call_count + 1
        if state.call_count == 1:
            raise ConnectionError("search index temporarily unavailable")
        lowered = query.lower()
        for topic in COMPANY_DOCS:
            if topic in lowered:
                return COMPANY_DOCS[topic]
        return "No matching document found."

    return search_docs
```

`agent.py` already takes an optional `tools` list (Approach 1's code above), so a test can pass the fresh tool in.

```
# test_agent.py
# Run it with pytest, from inside this folder: pytest test_agent.py -v
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from agent import run_agent
from docs_search import FlakyToolState, make_flaky_search_docs
from exceptions import MaxIterationsExceeded
from tools import add, get_weather


def test_clear_task_uses_the_tool() -> None:
```

```
    result = run_agent("What's 5 + 7?", 5)
    assert len(result.steps) >= 1
    assert result.steps[0]["tool"] == "add"


def test_no_tool_needed() -> None:
    result = run_agent("What's the capital of France?", 5)
    assert len(result.steps) == 0


def test_recovers_from_a_failure() -> None:
    # how: a fresh fail-once tool — its counter starts at 0 for this test
    search_docs = make_flaky_search_docs(FlakyToolState())
    tools = [add, get_weather, search_docs]
    result = run_agent("What is the refund policy?", 5, tools)
    failed = 0
    succeeded = 0
    for step in result.steps:
        if step["error"] is None:
            succeeded = succeeded + 1
        else:
            failed = failed + 1
    assert failed >= 1
    assert succeeded >= 1
    assert result.final_answer != ""


def test_step_limit_raises_with_the_log() -> None:
    try:
        run_agent("Add 1 + 2, then add 3 to that.", 1)
        assert False, "expected MaxIterationsExceeded"
    except MaxIterationsExceeded as e:
        assert len(e.steps) == 1
```

Run it: `pytest test_agent.py -v`, from inside `practice/build_task/`.

Expected output (shortened):

```
test_agent.py::test_clear_task_uses_the_tool PASSED
test_agent.py::test_no_tool_needed PASSED
test_agent.py::test_recovers_from_a_failure PASSED
test_agent.py::test_step_limit_raises_with_the_log PASSED
4 passed
```

Back to the Build Task · Hint 1 · Hint 2 · Solution

Approach 3 — arguments checked by your own Pydantic models, before any tool runs

Story — `run_tool()` with `ARG_MODELS`: the Intermediate exercise's Approach 4, applied to every tool: each tool's arguments must fit its Pydantic model before `.invoke()` is called, and `model_dump()` passes the checked values on. **If not:** the "checked before the tool runs" rule would rest only on each tool's own `args_schema`.

```
# agent.py
from dataclasses import dataclass, field

from langchain_core.tools import BaseTool
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, ValidationError

from docs_search import search_docs
from exceptions import MaxIterationsExceeded
from logging_setup import get_logger
from tools import AddArgs, WeatherArgs, add, get_weather

# why: Doc06's tools, reused unchanged, plus the one that fails once
AGENT_TOOLS = [add, get_weather, search_docs]


class SearchArgs(BaseModel):
    query: str


# how: tool name -> the Pydantic model its arguments must fit
ARG_MODELS = {
    "add": AddArgs,
    "get_weather": WeatherArgs,
    "search_docs": SearchArgs,
}


@dataclass
class AgentResult:
    final_answer: str
    # why: a list default needs field(default_factory=list), not `= []`
    steps: list[dict] = field(default_factory=list)
```

```
def find_tool(tools: list[BaseTool], name: str) -> BaseTool | None:
    # how: Doc06's Build Task function, copied unchanged
    for candidate in tools:
        if candidate.name == name:
            return candidate
    return None


def run_tool(
    call: dict, tools: list[BaseTool]
) -> tuple[str | None, str | None]:
    tool = find_tool(tools, call["name"])
    if tool is None:
        return None, f"unknown tool: {call['name']}"
    try:
        if call["name"] in ARG_MODELS:
            # why: the README's rule, literally — the arguments are
            # checked BEFORE the tool runs (Intermediate exercise,
            # Approach 4)
            checked = ARG_MODELS[call["name"]](**call["args"])
            result = tool.invoke(checked.model_dump())
        else:
            result = tool.invoke(call["args"])
        return str(result), None
    except ValidationError:
        return None, f"invalid arguments for {call['name']}: {call['args']}"
    except Exception as e:
        return None, str(e)


def run_agent(
    task: str, max_iterations: int, tools: list[BaseTool] | None = None
) -> AgentResult:
    # why: None means "use AGENT_TOOLS"; a test can pass its own tools
    if tools is None:
        tools = AGENT_TOOLS
    logger = get_logger(__name__)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools(tools)
    messages = [{"role": "user", "content": task}]
    steps: list[dict] = []

    for step in range(max_iterations):
        response = model_with_tools.invoke(messages)
        messages.append(response)
        if not response.tool_calls:
            return AgentResult(final_answer=response.content, steps=steps)
```

```
        # why: run EVERY call the model asked for — each needs an answer
        for call in response.tool_calls:
            result, error = run_tool(call, tools)
            if error is None:
                observation = result
                logger.info(f"step {step}: {call['name']} -> {result}")
            else:
                observation = f"Error: {error}"
                # how: a warning, not an error — the loop keeps going
                logger.warning(f"step {step}: {call['name']} failed: {error}")
            messages.append({
                "role": "tool",
                "content": observation,
                "tool_call_id": call["id"],
            })
            steps.append({
                "step": step,
                "tool": call["name"],
                "arguments": call["args"],
                "result": result,
                "error": error,
            })

    message = f"No answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)
```

Expected output: `test_agent.py` prints the same 4 `case` lines as Approach 1. A bad call — `{"name": "add", "args": {"a": "two", "b": 1}}` — now returns `(None, "invalid arguments for add: {'a': 'two', 'b': 1}")` without `add` ever running.

Which one should you actually use?

Approach 1 meets every requirement: 3 tools, Pydantic-checked arguments, a hard step limit, recovery from a real failure, and a full step log — Project 2's Worker is this same loop. Add Approach 2's pytest tests once you use pytest (Doc13), and Approach 3's `ARG_MODELS` when you want your own error message for bad arguments.